

React Learning — Session Summary

Topics Covered Today:

1. State ownership in React
2. Lifting state up
3. When not to lift state
4. Global state management (Context, Redux, Zustand, Jotai)

1. State Belongs to a Component

- Each component **owns its own state**.
- Example:

```
function App() {  
  const [count, setCount] = useState(0);  
}
```

- `count` is private to `App`.
- Other components can only read it via **props**.

Key Principles: - State is **local** to the component instance. - Keep state **close to where it is used**. - Avoid unnecessary re-renders by not lifting state too early.

2. Lifting State Up

- Needed when **multiple components need shared state**.

Example:

```
function Calculator() {  
  const [temperature, setTemperature] = useState("");  
  
  return (  
    <div>  
      <TemperatureInput temperature={temperature} onChange={setTemperature} />  
      <BoilingVerdict temperature={temperature} />  
    </div>  
  );  
}
```

- Parent component (`Calculator`) owns the state.

- Child components receive state via **props** and notify changes via **callbacks**.

Data Flow:

```
Parent (state owner)
├─ Child1 (props + callback)
└─ Child2 (props)
```

Key Rule:

“In React, the component that needs the data should own the state.”

3. When NOT to Lift State

- Keep state **local** if only one component needs it.

Example:

```
function ToggleButton() {
  const [on, setOn] = useState(false);

  return <button onClick={() => setOn(!on)}>{on ? "ON" : "OFF"}</button>;
}
```

- Over-lifting leads to **unnecessary re-renders**.
- Use **React.memo** or derived state patterns if lifting is unavoidable.

Takeaway:

Lift state **only when necessary**, otherwise keep it local.

4. Global State in Large Apps

- **Prop drilling** problem arises when state must be shared deeply:

```
<App>
  <Content>
    <ProductList>
      <ProductCard />
    </ProductList>
```

```
    </Content>
  </App>
```

- Passing props through layers is cumbersome.

Solutions:

4.1 Context API

```
const UserContext = createContext();

function App() {
  const [user, setUser] = useState({ name: "Faisal" });

  return (
    <UserContext.Provider value={user}>
      <Header />
      <Content />
    </UserContext.Provider>
  );
}

function ProductCard() {
  const user = useContext(UserContext);
  return <p>Hello, {user.name}</p>;
}
```

- Medium apps: works well, avoids prop drilling.

4.2 Redux / Zustand / Jotai

- Large apps with complex interactions need advanced state management.
- Features:
 - Any component can read/write state directly
 - Avoid prop drilling
 - Fine-grained performance optimizations

Key Principle:

Lift state → If prop drilling becomes messy → Use Context or global state library.

5. Session Summary / Key Takeaways

1. State **belongs to the component** that uses it.

2. Lift state up **only when multiple components need it**.
3. Keep state **local** whenever possible to avoid unnecessary renders.
4. For large apps, use **Context or global state libraries** to manage shared state efficiently.